

Q1 A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

1) Representation in 8-bit 2's complement:

$$+45: \quad 45_{10} = 0010 \ 1101_2$$

$$-23: \quad 23_{10} = 0001 \ 0111_2$$

2's complement of 23:

$$\text{Invert bits} : \quad 1110 \ 1000$$

$$\text{Add 1} : \quad 1110 \ 1001 \rightarrow -23$$

2) Addition: $+45 + (-23)$

$$\begin{array}{r} 0010 \ 1101 \ (+45) \\ + \ 1110 \ 1001 \ (-23) \\ \hline 1 \ 0001 \ 0110 \end{array}$$

(Discard carry out)

$$0001 \ 0110 = +22$$

3) Subtraction: $+45 + (-23) = 45 + 23$

$$23 \text{ in binary} : \quad 0000 \ 0111$$

$$2's \text{ complement of } (-23) = 1110 \ 1001$$

$$\begin{array}{r} 0010 \ 1101 \ (+45) \\ + \ 0001 \ 0111 \ (+23) \\ \hline 0100 \ 0100 \end{array}$$

$$0100 \ 0100 = +68$$

4) Overflow Detection:

In 2's complement addition, overflow occurs if the two operands have the same sign and the result has a different sign. This can be detected by checking the carry into the sign bit (C_{n-1}) and the carry out of the sign bit (C_n). If $C_{n-1} \neq C_n$, overflow occurs.

• For +45 and -23: Operands have different signs $\rightarrow$ overflows cannot occur.

• For +45 and +23: Operands have same sign and result (+68) is also positive $\rightarrow$ No overflow.

Maximum positive in 8-bit 2's complement = +127

Minimum negative = -128. Both results (22 and 68) are within this range.

$\therefore$ No overflow occurs in either operation.

$$\text{Final Results: } +45 + (-23) = +22, \quad +45 - (-23) = +68$$

Q2 A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Let,

$$A = A_3 A_2 A_1 A_0 \quad \text{and} \quad B = B_3 B_2 B_1 B_0$$

C_{in} = Carry input

1) Generate (G) and Propagate (P) signals:

For each bit i ($i = 0$ to 3)

$$G_i = A_i B_i \quad (\text{Generate})$$

$$P_i = A_i \oplus B_i \quad (\text{Propagate})$$

2) Carry Look-Ahead Equations:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

3) Sum outputs:

$$S_i = P_i \oplus C_i \quad \text{for } i = 0 \text{ to } 3$$

4) Example

$$\text{Let, } A = 1011, B = 0110, C_{in} = 0$$

Bit (i)	A_i	B_i	$G_i = A_i B_i$	$P_i = A_i \oplus B_i$
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Carry Computation

$$C_0 = 0$$

$$C_1 = G_0 + P_0 C_0 = 0 + 1 \cdot 0 = 0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 1 + 0 + 0 = 1$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 = 0 + 0 + 0 + 0 = 0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \\ = 0 + 0 + 0 + 0 + 0 = 0$$

Sum

i	3	2	1	0
P_i	1	1	0	1
C_i	0	0	1	0
$S_i = P_i \oplus C_i$	1	1	1	1

$$\text{Result} = 1111 \quad (15)_{10}$$

5) Delay Comparison:

• Ripple Carry Adder (RCA):

Carry must propagate through each full adder sequentially.

Delay $\propto$ (Linear)

• Carry Look-Ahead Adder (CLA):

Carries are computed in advance using Boolean equations

Delay $\propto \log_2 n$ (very small)

$\therefore$ CLA is faster than RCA because it reduces carry propagation time, which is the major source of delay in arithmetic circuits. Hence, CLA is widely used in modern high-speed processors.

Q3 In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Let, Multiplicand (M) = -6 , Multiplier (Q) = $+5$

In 4-bit 2's complement:

$$M = -6 \rightarrow 1010$$

$$Q = +5 \rightarrow 0101$$

Initialize: $A = 0000$, $Q = 0101$, $Q_{-1} = 0$, $n = 4$

Step	Q_n	Q_{-1}	Operation	A (after op)	After Arithmetic Right Shift		
					A	Q	Q_{-1}
Initial	-	-	-	0000	0000	0101	0
1	1	0	$A = A - M$ ($A = A + 6$)	0110	0011	0010	1
2	0	1	$A = A + M$ ($A = A - 6$)	1101	1110	1001	0
3	1	0	$A = A - M$ ($A = A + 6$)	0100	0010	0100	1
4	0	1	$A = A + M$ ($A = A - 6$)	1000	1100	0010	0

After n steps, the product is in $(A.Q) = 1100\ 0010$

This is an 8-bit 2's complement number

$1100\ 0010 \rightarrow$ Negative number

2's complement $\rightarrow$ Invert: $0011\ 1101$, add 1 $\rightarrow 0011\ 1110 = 62$

Hence, $1100\ 0010 = -62$

Final Product: $(-6) \times (+5) = -30_{10}$ (Decimal)

Explanation:

Booth's algorithm looks at (Q_n, Q_{-1}) pair. Depending on the pair, it performs either addition, subtraction or no operation. This reduces the number of additions/subtractions, especially when there are consecutive 1's in the multiplier. It works efficiently for signed numbers using arithmetic right shifts, preserving the sign.

Q4 A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Let, Dividend = 13 (1101)₂ , Divisor = 3 (0011)₂

(A) Restoring Division

Step	A (Remainder)	Q (Quotient)	Operation
Init	0000	1101	-
1	0001	1010	Shift left, Subtract (0001 - 0011 = -0010) → Restore
2	0011	0100	Shift left, Subtract (0011 - 0011 = 0000) → Set $Q_2 = 1$
3	0011	1000	Shift left, Subtract (0011 - 0011 = -0010) → Restore
4	0010	0000	Shift left, Subtract (0010 - 0011 = -0001) → Restore

Quotient = 0100 (4)₂ , Remainder = 0010 (2)₂

(B) Non-Restoring Division

Step	A (Remainder)	Q (Quotient)	Operation
Init	0000	1101	-
1	0001	1010	Shift left, $A = A - M$ (0001 - 0011 = -0010)
2	0000	0100	Shift left, $A = A - M$ (-0010 + 0011 = 0001)
3	0011	1000	Shift left, $A = A - M$ (0001 - 0011 = -0010)
4	0001	0000	Shift left, $A = A - M$ (-0010 + 0011 = 0001)

Final correction ($A > 0$): $A = A - M = 0001 - 0011 = -0010$

Quotient = 0100 (4)₂ , Remainder = 0010 (2)₂

Comparison	Restoring Division	Non-Restoring Division
Operation per step	May require restore step	No restore step, only add/sub
Number of operations	More (add, subtract and restore)	Less (only add or subtract)
Speed	Slower	Faster
Hardware	More control logic	Less control logic
Efficiency	Less efficient	More efficient

∴ Non-restoring division is preferred in modern systems because it requires fewer operations and less hardware, hence faster.

Q5 A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent and mantissa fields. Further, explain how floating point addition is performed between two such numbers, highlighting issues like normalization, rounding and precision loss.

1) Representing -13.25 in IEEE 754

Convert 13.25 to binary

13 = 1101, 0.25 = 0.01, → 13.25 = 1101.01.

Normal: $110101_2 = 1 \cdot 10101_2 = 2^1$

Sign bit, $S = 1$ (negative number)Experimental, $E = 3$
$$\text{Biased exponent} = E + \text{Bias}$$

- For single precision (Bias = 127) $3 + 127 = 130 = 1000\ 0010_2$
- For double precision (Bias = 1023) $3 + 1023 = 1026 = 100\ 0000\ 0010_2$

Mantissa = fractional part after leading 1 $\rightarrow$ 10101

Single Precision (32-bit)

S (1 bit)	Exponent (8 bits)	Mantissa (23 bits)
1	1000 0010	:0101 00000000000000000000000

Hex: C141 4000

Example: *Phragmites* (44-46)

[illegible]

Hex: C07A 8000 0000 0000

2) Floating-Point Addition (Example)

 $\Delta_{\text{ad}} = 14.35 \text{ kcal} \cdot \text{mol}^{-1}$

- (i) Convert to binary and normalize
- $$-15.25 = -1.10101 \times 2^4 \quad , \quad +7.5 = +1.111 \times 2^3$$

- (u) Align exponents (use larger exponent = 3)

- (ii) Add mantissas (consider sign)
- $$\begin{array}{r} -1.10101 \\ + 0.11110 \\ \hline -0.10111 = 2^{-1} \end{array}$$

- (iv) Normalized result
 $-0.102111 \times 10^3 = -1.02111 \times 10^2$

- (v) Round to the mantissa bits (e.g., single precision has 23 bits) of accuracy.

Final Result: $(-13.35) + (+7.5) = -5.85$

IEEE 754 standard allows wide range and precision, but due to finite bits, rounding and precision loss are inevitable

Issues in Floating Point Arithmetic

- Normalization: After addition, result may need to be normalized to the form $1 \text{ case} \times 2^k$.
- Rounding: Since mantissa has limited bits, extra bits are rounded off $\rightarrow$ may cause small errors.
- Precision Loss: Numbers with very different exponents may lose precision in smaller terms during alignment.